



아키텍처 다이어그램

[1] 아키텍처 다이어그램

1. 정의
2. 왜 그리는가?
3. 다이어그램에 무엇을 나타낼까? - 관점과 추상화 수준을 결정해야
 - 관점 — 무엇을 보여줄까
 - 추상화 수준 — 얼마나 자세히: C4 모델
 - 같이 보면 이렇게
4. 작성 절차
 - 준비
 - 작성
 - 검토
5. 흔한 실수 — 진단 체크리스트
6. 스터디 표기 컨벤션
7. 마무리

[2] 아키텍처 다이어그램 그리기 도구

- draw.io - 가장 대중적
- visual paradigm - 아이콘이 많다
- Excalidraw - 낙서 느낌**
- Lucidchart - 플로우차트 느낌

[3] 다이어그램을 그릴 때 우리 스터디의 원칙

- 디자인 중요하지 X
- AI 자동화?

[4] 참고자료 및 부록

- 참고자료
- 부록

[1] 아키텍처 다이어그램

1. 정의

아키텍처 다이어그램: **특정 청중에게 시스템의 특정 측면을 설명하기 위한 시각 모델**

특징

- **청중 의존성:** 누구에게(ex.경영진·기술팀·세일즈·신규 입사자) 보여줄 것이냐에 따라 추상화 수준이 달라짐.
- **부분성:** 한 장의 다이어그램으로 시스템 전체를 담을 수 없음. 어떤 측면에 집중하면 나머지는 생략해야 함.

2. 왜 그리는가?

이득

- 협업: 통합된 시스템 뷰를 공유해 합의 비용을 낮춤
- 의사결정 지원: 기술 선택과 자원 할당의 근거를 제공
- 잘못된 설계 위험 감소: SDLC 초기에 잘못된 가정과 결함 발견 가능
- 성능 최적화: 병목·확장 지점·장애 전파 경로 파악

그리지 않는다면? 🤔

- 구조가 한번에 파악되지 않아 잘못된 부분을 찾기 어려움
- 신규 합류 팀원의 온보딩 비용 누적
- 회의에서 아키텍처를 보며 얘기할 때 매번 다시 그려야

3. 다이어그램에 무엇을 나타낼까? - 관점과 추상화 수준을 결정해야

다이어그램은 하나의 시스템에 대해 여러 장이 필요함. 한장으로 전체 표현 불가.

관점 — 무엇을 보여줄까

유형	보여주는 것
시스템	컴포넌트 구조와 상호작용
애플리케이션	앱 내부 구조와 데이터 흐름
통합	시스템 간 연결 관계
배포	서버·스토리지·네트워크 배치
DevOps	CI/CD, IaC, 클라우드 서비스
시퀀스	컴포넌트 간 요청-응답 왕복 통신

추상화 수준 — 얼마나 자세히: C4 모델

- **Context:** 시스템과 외부 사용자·시스템의 관계
- **Container:** 시스템 내부의 배포 가능 단위 (앱, DB, 서비스)
- **Component:** 컨테이너 내부 구성 요소
- **Code:** 클래스·함수 수준 (보통 생략)

같이 보면 이렇게

	Context	Container	Component
시스템	○	○	
배포		○	
시퀀스		○	○

다이어그램에서 선택이 중요하다는 것을 알겠죠?

4. 작성 절차

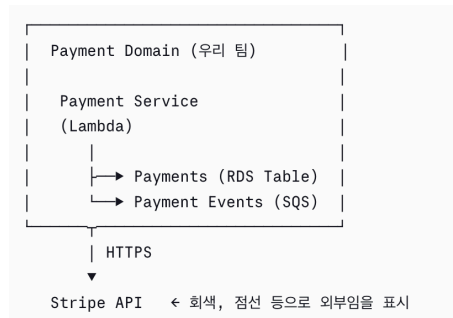
준비

- 목적, 청중 명확화 (1다이어그램 1목적)
 - 저희는 엔지니어 팀원을 청중으로 잡으면 될 것 같습니다.
- 정보 수집: 컴포넌트, 데이터 흐름, 인터페이스, 의존성
- 도구 선택: draw.io, Excalidraw, PlantUML, Mermaid 등

작성

(1) 시스템 경계 정의

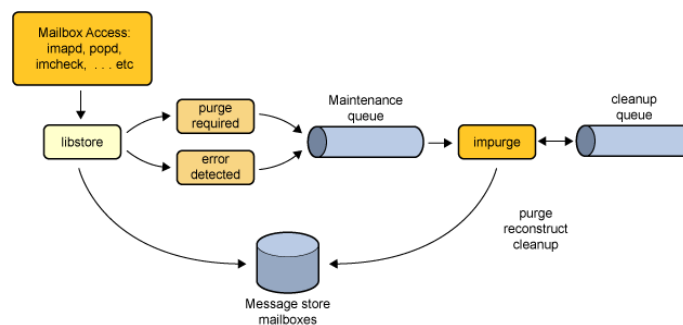
- 다이어그램의 주인공(우리가 설계·운영하는 부분)을 **박스**로 묶음. 박스 제목에 도메인 이름 표기.
- 박스 안 = 내부 (자세히 설명할 대상, 색깔로 강조)
- 박스 밖 = 외부 의존 (우리가 제어 X, 인터페이스만 노출된 것)
 - 사내 다른 팀의 서비스라도 직접 코드를 고칠 수 없다면 외부



(2) 도형으로 컴포넌트 표현

한 다이어그램 안에서 같은 종류는 같은 도형으로 통일

- 일반 서비스·애플리케이션: 사각형
 - 컴포넌트가 포함 관계다: 안에 넣기
- 데이터베이스·스토리지: 원통
- 외부 사용자·액터: 원/타원
- 큐: 누운 원통, 원통



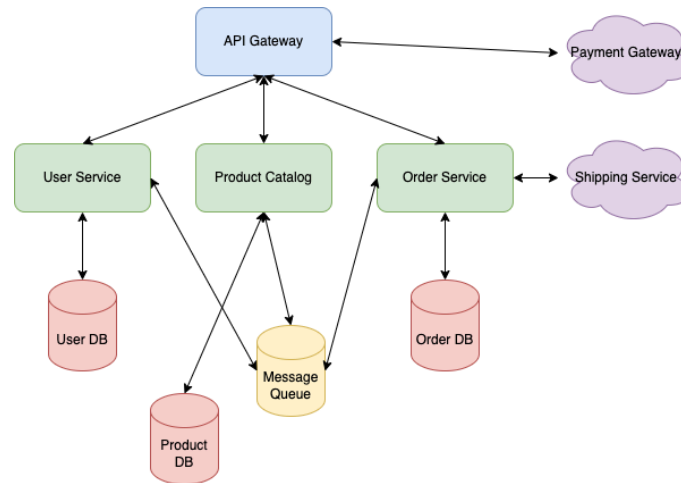
(3) 컴포넌트 배치

- 데이터·요청 흐름이 위 → 아래 또는 왼쪽 → 오른쪽으로 흐르도록 배치 (쪽 보기 좋게)
- 같은 계층(예: API 계층, 데이터 계층)은 가로로 정렬
- 외부 사용자/시스템은 가장자리에 배치
- 선이 가급적 교차하지 않도록 위치 조정 — 교차가 많으면 뭔가 잘못됐다. 배치 자체를 바꿔보자

(4) 레이블 — 이름과 타입을 병기하는 것이 좋다

- 이름: 그 컴포넌트의 고유한 역할 (예: `Orders`, `User Auth`, `Results`)
- 타입: 무엇인지 (예: `Service`, `Table`, `Bucket`, `Queue`, `Lambda`, `RDS`)

Database가 여러개인데, 레플리카 목적도 아닌데 "Database 1", "Database 2", "Database 3"
→ 무슨 용이지...? 🤔



아하! 한눈에 알겠다!

(5) 화살표 — 제어 or 데이터 흐름

- 화살표: 제어 흐름(누가 누구를 호출하는지 요청 방향) 또는 데이터 이동 흐름 나타냄
 - 한 다이어그램 안에서는 하나만 나타내는 것이 좋음. 흐름을 자세히 하고 싶으면 시퀀스 다이어그램을 별도로 작성
- 화살표 위에 통신 방식, 프로토콜·동기/비동기 표시 (예: `HTTPS`, `gRPC`, `async`)

(6) 효율적인 협업을 위해선 범례를 정해두는 것이 좋다

- 도형 의미, 색 의미, 화살표 의미, 점선/실선 기준..

검토

- 정확성·완전성 점검. 누락된 연결 보완

5. 흔한 실수 — 진단 체크리스트

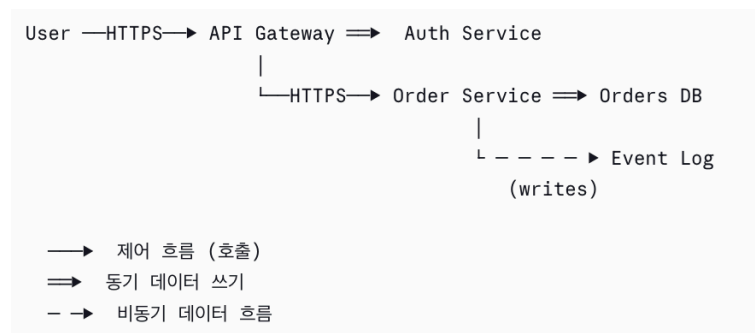
증상	원인	해결
봐도 무슨 시스템인지 모르겠음	청중과 목적이 안 정해짐	한 다이어그램에 한 가지 목적만
박스에 타입만 적혀 있어 정체 파악 불가	이름 빠짐	이름과 타입 같이 적기
어떤 컴포넌트가 왜 거기 있는지 설명 못 함	최종 설계에서는 필요 없어짐	빼거나 역할 적기
한 장에 다 그려서 안 읽힘	욕심을 많이 내서 마스터 다이어그램이 됨	관점·추상화 수준으로 쪼개기

증상	원인	해결
왕복 통신인데 일방향처럼 보임	시퀀스가 빠짐	시퀀스 다이어그램 같이 그리기
이벤트 브로커 뒤에서 누가 누구를 구독하는지 사라짐	팬 트랩 (Fan Trap)	토픽 단위까지 드러내기
최신 상태가 반영 안 됨	고치기 번거로운 형태로 만들어짐	코드처럼 버전 관리 (PlantUML, Mermaid)

6. 스터디 표기 컨벤션

발표 일관성을 위해 아래 규칙을 참고해 주시면 좋을 것 같아요

- **레이블:** 이름, 타입 명확히 (ex. `Auth Server`)
- **도형 규칙:**
 - 사각형 = 서비스/애플리케이션
 - **원통 = DB/스토리지**
 - 타원 = 외부 사용자/액터
 - 박스 = 시스템 경계 또는 논리적 그룹
 - CSP 아이콘 활용하셔도 됨
- **화살표:** "요청 흐름" 또는 "데이터 이동 흐름"
 - 통신하는 경우 프로토콜 표기
 - 요청과 데이터 중 하나만 나타내는 것이 좋음. 둘 다 나타내야 한다면 **화살표 디자인을 구분**. (ex: 선 스타일을 다르게 / 색을 다르게 / 머리 모양을 다르게)



- **외부 의존:** 시스템 경계 밖에 두기

금지

- 어디에도 연결되지 않은 컴포넌트 → 삭제할 것
- 이름, 의미 없이 타입만 (`EC2`, `Lambda`) → 의미 이해되게

7. 마무리

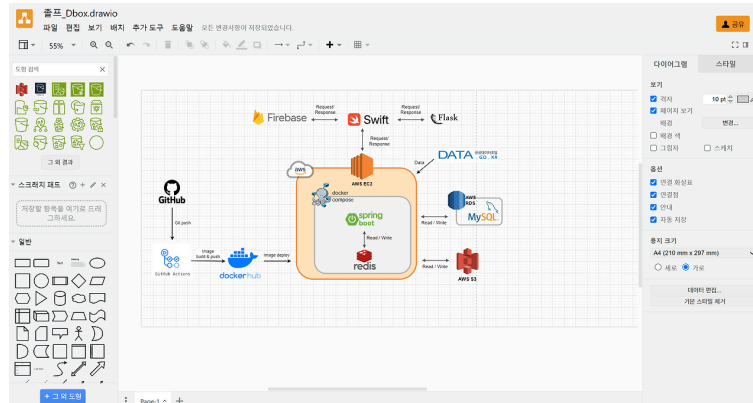
좋은 다이어그램의 기준은 "예쁘다"가 X.

다이어그램을 보고 의사결정을 내릴 수 있어야 좋은 다이어그램.

[2] 아키텍처 다이어그램 그리기 도구

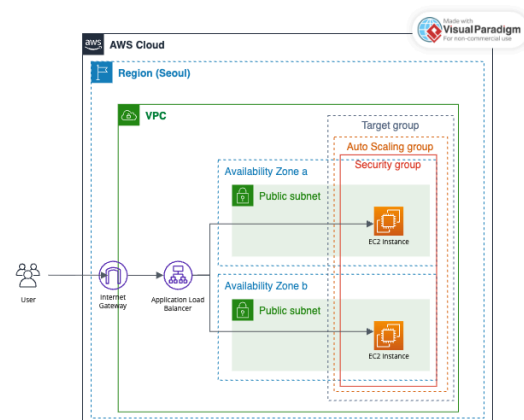
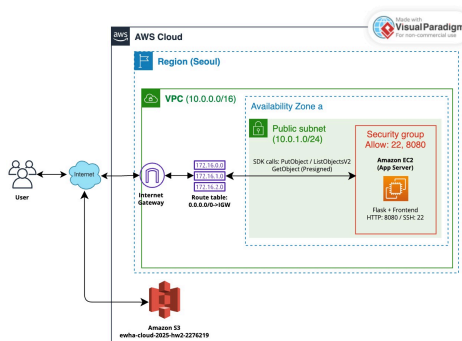
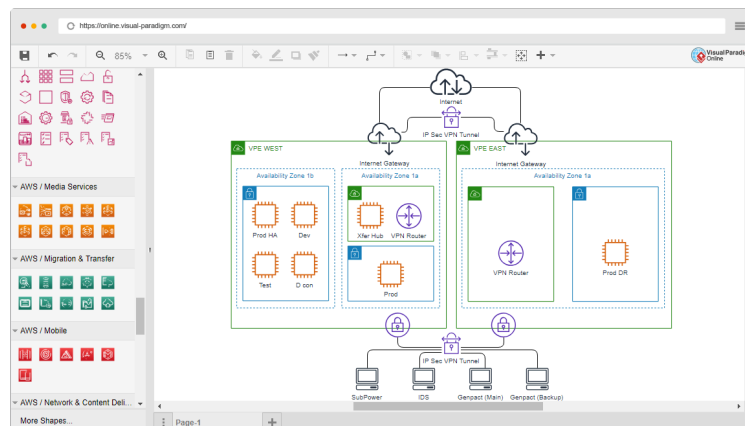
draw.io - 가장 대중적

<https://app.diagrams.net/>



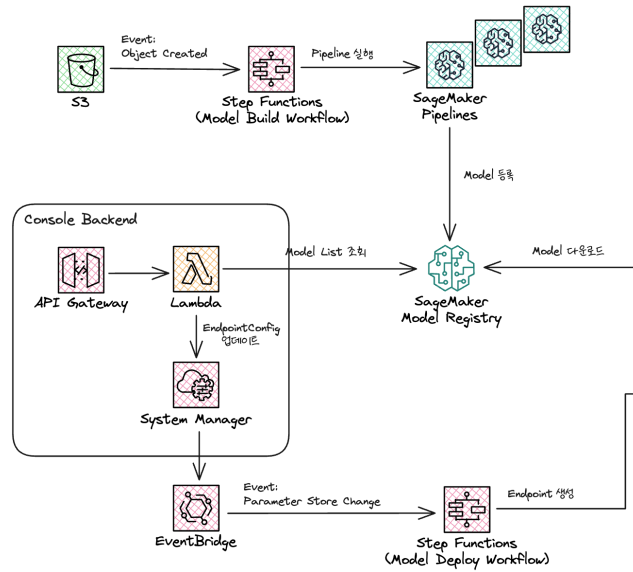
visual paradigm - 아이콘이 많다

<https://online.visual-paradigm.com/>



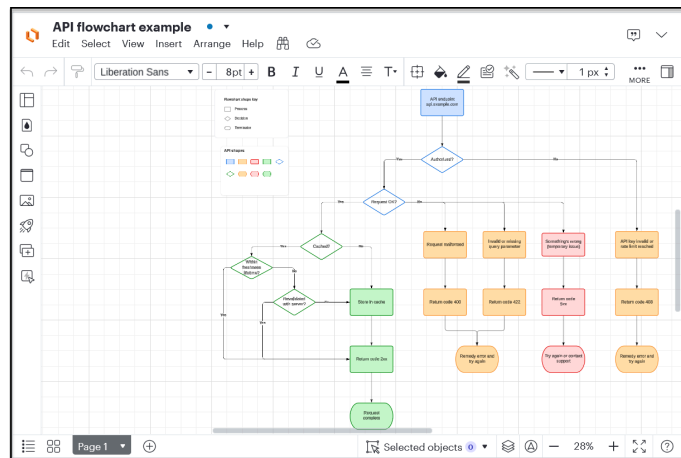
Excalidraw - 낙서 느낌

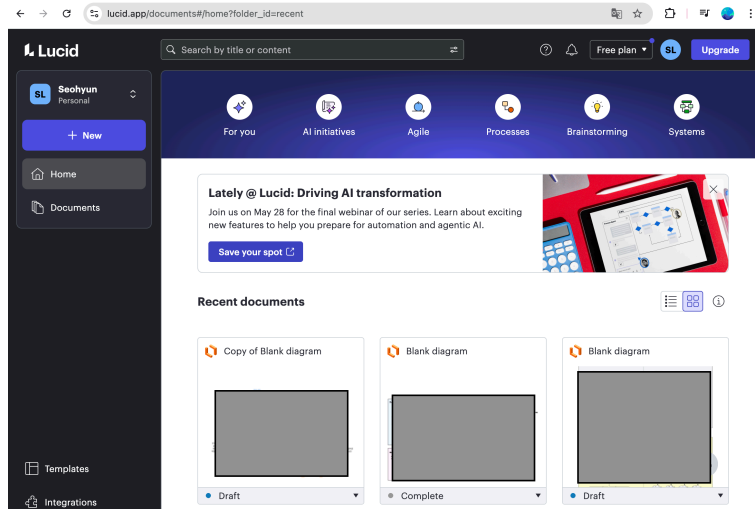
<https://devocean.sk.com/blog/techBoardDetail.do?ID=165882&boardType=techBlog>



Lucidchart - 플로우차트 느낌

<https://lucid.app/>





[3] 다이어그램을 그릴 때 우리 스터디의 원칙

디자인 중요하지 X

- 색깔 중요 X. 아이콘 중요 X. 알아볼 수 있으면 됩니다.
- 손으로 그려도 됩니다.

AI 자동화?

- **AI로 “아키텍처 이미지 생성” 금지. 수정이 불가능해서.**
 - 저도 GPT Image 2.0로 생성 한번 해봤는데 화살표가 이상한 위치에 그려졌어요.
- mermaid 코드 생성하고 렌더링해 제출하는 것은 허용.
 - 그래도 도형, 이미지에 한계가 있고 모듈이 너무 많으면 가독성이 낮아서 비추.
- AI를 활용하고 싶다면 draw io 파일로 달라고 한 뒤 불러와서 깔끔하게 수정하는 시도 한번 해보시길 추천
 - 사실 저도 안 해봤는데 어느 분이 이렇게 하시는 경우를 봤어요

[4] 참고자료 및 부록

참고자료

<https://aws.amazon.com/ko/what-is/architecture-diagramming/>
<https://www.edrawsoft.com/kr/diagram-tutorial/c4-model-guide.html?srsId=AfmBOoqkhzqm5dhCdGx97B8-ji8TxzbsMM-CiPCW0EtOahXZT0I0qsX>
<https://www.processon.io/ko/blog/architecture-diagram-making-tutorial>
<https://wikidocs.net/blog/@jaehong/9751/>

Claude

부록

- 자동화 관련 글 (저도 다 읽어보지는 않았어요)

AI로 아키텍처 다이어그램 그리기

정도현 - 로보코 수석 컨설턴트 바이브 코딩을 할 때, 사람들은 종종 코드로만 충분하다고 생각한다. 하지만 현실은 다르다. 문서화 없는 코드는 마치 지도 없이 떠나는 여행과 같다. 목적지가 어딘지, 가는 길이 어떤지 알 수 없다면, 누구도 함께하지 않을 것이다. 바이브 코딩에서 문서화가 특히 중요한 이유는 코드가 단지 작동하기 위한 것이 아니라, 아이디어와 설계의 의도를 다른 사람과 정확히 공유하기 위한 것이기 때문이다. 문서화는 협업을

 <https://roboco.io/posts/how-to-draw-diagram/>

How to use Claude AI + draw.io to Create Architecture Diagrams for Projects

Introduction In technical engineering projects, especially those involving software system...

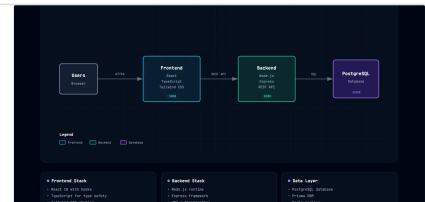
 <https://dev.to/rushier/how-to-use-claude-ai-drawio-to-create-architecture-diagrams-for-projects-17i1>



아키텍처 다이어그램, 이제 말로 설명하면 AI가 그려준다

시스템 구조도를 그리는 일은 개발자에게 묘한 숙제다. 설계는 머릿속에 있는데, 이것을 깔끔한 그림으로 옮기려면 Lucidchart나 draw.io 같은 도구를 열어 박스를 정렬하고 화살표를 연결하며 색을 맞추는 데 시간을 쏟아야 한다. 정작 전달하고 싶은 건 구조의 논리인데, 노동의 대...

 <https://wikidocs.net/blog/@jaehong/12011/>



[reinvent 2024] 생성형 AI를 활용해 아키텍처 다이어그램을 생성하고 이해하기

이 세션에서는 이미지, draw.io, 텍스트 등으로 존재하는 아키텍처 다이어그램을 생성형 AI를 사용하여 일관된 형식의 아키텍처 다이어그램을 생성하는 방법을 공유합니다. Visual Studio Code에서 Amazon Q developer를 사용하여 다양한 형식의 아키텍처 다이어그램을 생성할 수

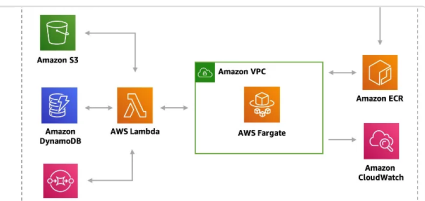
 <https://iting.co.kr/reinvent-techblog-2024-post-13/>



시스템 아키텍처 다이어그램, 아직도 이렇게 그리고 있다면 다시 생각해보자

소프트웨어 시스템이 복잡해질수록 아키텍처 다이어그램의 중요성은 커진다. 그런데 정작 그 다이어그램이 소통을 돕기는커녕 혼란을 가중시키는 경우가 놀랍도록 많다. Ilograph의 Billy Pilger가 정리한 아키텍처 다이어그램에서 흔히 저지르는 7가지 실수 후속편이 HN에서 활발한

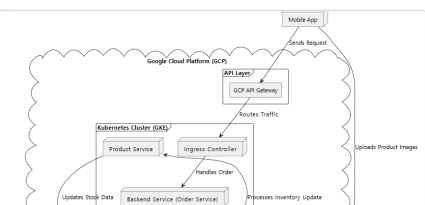
 <https://wikidocs.net/blog/@jaehong/9751/>



챗GPT로 아키텍처 그리기

기술 회의와 디스커버리 워크숍을 진행하다 보면 시스템 아키텍처를 그려야 할 때가 많습니다. '설명하지 말고 보여주라'는 말처럼, 아키텍처는 고객을 이해시키는 데 매우 효과적인 도구입니다. 특히 기술 회의 중에 고객의 질의를 반영해 즉시 아키텍처를 그릴 수 있으면 정말 유용하죠.

 <https://brunch.co.kr/@bromong/77>



Claude가 Excalidraw를 직접 그려준다고? — 공식 MCP 커넥터

지난 이야기 <https://for-habit.tistory.com/154> 아키텍처 다이어그램, 매번 손으로 그리기 싫어서 자동화해본 이야기발단: Excalidraw는 좋은데 매번 그리기엔 너무 귀찮다우리 팀은 아키텍처 설계하거나 블로그 쓸 때 항상 Excalidraw로 그림을 그린다. 깔끔하고 손그림 느낌도 나고, 원가

 <https://for-habit.tistory.com/155>

